CAPSTONE PROJECT – TECHNICAL SUBMISSION

INDUSTRIAL IoT AUTOMATION SYSTEM

USING ESP32 AND HIVEMQ CLOUD FOR REMOTE MONITORING AND CONTROL

Course: ECA1407 – Embedded Systems

Degree: Bachelor of Engineering in Electronics and Communication Engineering

Student: Paul David – 192572144

Supervisor: Dr. Yuvaraj

Institution
SIMATS Engineering
Saveetha Institute of Medical and Technical Sciences
Chennai – 602105

Technical implementation package prepared from the submitted project record and the supplied technical submission requirements.

TECHNICAL SUBMISSION PACKAGE

This document is organized to directly address the evaluator's required technical package: circuit/system schematic, source code, setup and execution instructions, experimental setup, experimental data, testing and validation, project evidence, and final submission links. The supplied instructions require enough information for an evaluator to understand, reproduce, run, test and verify the project without additional clarification.

Section	Included content
1. Project identification	Project title, purpose, scope and implementation summary
2. Circuit / system schematic	System architecture, detailed interconnections, GPIO mapping and MQTT interfaces
3. Hardware and software requirements	Components, development tools and libraries
4. Experimental setup	Actual prototype photograph, arrangement, operating conditions and measurement method
5. Instructions to run	Hardware preparation, software installation, configuration, upload, operation and verification
6. Experimental data	Traceable documented snapshot, raw/processed data and evidence limitations
7. Testing and validation	Requirement-linked test plan and observed functional results
8. Results and engineering interpretation	Objective achievement, decision logic and limitations
9. Project evidence	Prototype and OLED photographs
10. Source code	Complete Arduino/ESP32 source with credentials represented as placeholders
11. Final links and checklist	GitHub/video placeholders and submission checklist

Evidence rule: No unsupported measurements have been invented. The project record explicitly states that repeated sensor, latency, packet-loss and long-duration reliability datasets are unavailable; therefore those values are identified as validation gaps rather than fabricated results.

1. PROJECT IDENTIFICATION & TECHNICAL SUMMARY

Project: Industrial IoT Automation System Using ESP32 and HiveMQ Cloud for Remote Monitoring and Control.

The prototype combines an ESP32 controller, DHT11 temperature/humidity sensor, MQ-2 analog gas/smoke-related sensor, 128 × 64 SSD1306 OLED, relay interface and demonstration fan/load. Wi-Fi provides network access and HiveMQ Cloud provides the MQTT broker path. The embedded controller performs local threshold-based fan control while also publishing telemetry and accepting remote MQTT commands.

The implemented automatic rule is: **FAN = ON when temperature > 30 °C OR MQ-2 ADC > 700** while AUTO mode is active. Manual MQTT commands support AUTO, ON and OFF. Sensor acquisition is scheduled every 3 seconds and MQTT reconnection is attempted at approximately 5-second intervals.

The design intentionally retains the automatic control loop locally so that basic actuation does not depend on continuous cloud availability. The MQ-2 reading is treated only as a raw ADC value; no calibrated gas concentration or certified safety-alarm capability is claimed.

Parameter	Implemented value / description
Controller	ESP32 development board
Temperature / humidity	DHT11
Gas/smoke-related input	MQ-2 analog output, raw ADC
Local display	0.96-inch SSD1306 OLED, 128 × 64
Actuator interface	Relay module controlling demonstration fan/load
Network	Wi-Fi
Cloud communication	MQTT through HiveMQ Cloud
Telemetry topic	industrial/site1/telemetry
Command topic	industrial/site1/command
Status topic	industrial/site1/status
AUTO threshold	Temperature > 30 °C OR gas ADC > 700
Sampling schedule	3000 ms nominal
MQTT retry schedule	5000 ms nominal

2. CIRCUIT DIAGRAM / SYSTEM SCHEMATIC

The project record provides both a system architecture and a simplified circuit/interconnection drawing. The connections below reproduce the documented GPIO assignments. Exact load current, relay contact rating, supply margin and detailed power wiring were not recorded in the supplied evidence, so those values are deliberately not invented here.

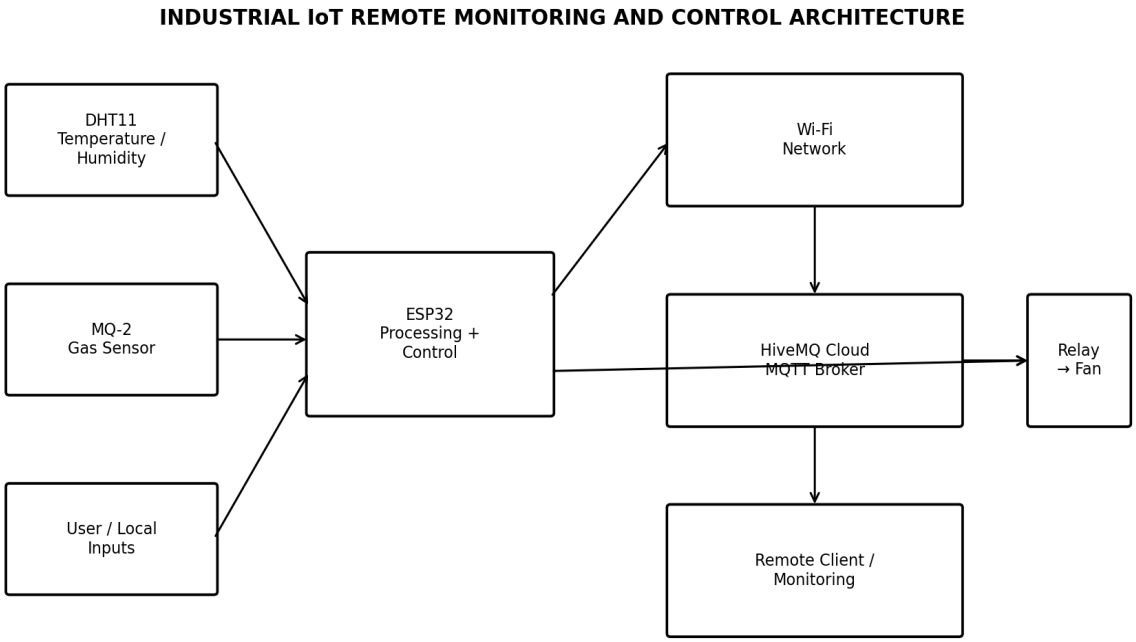

Figure 2.1 Industrial IoT remote monitoring and control architecture

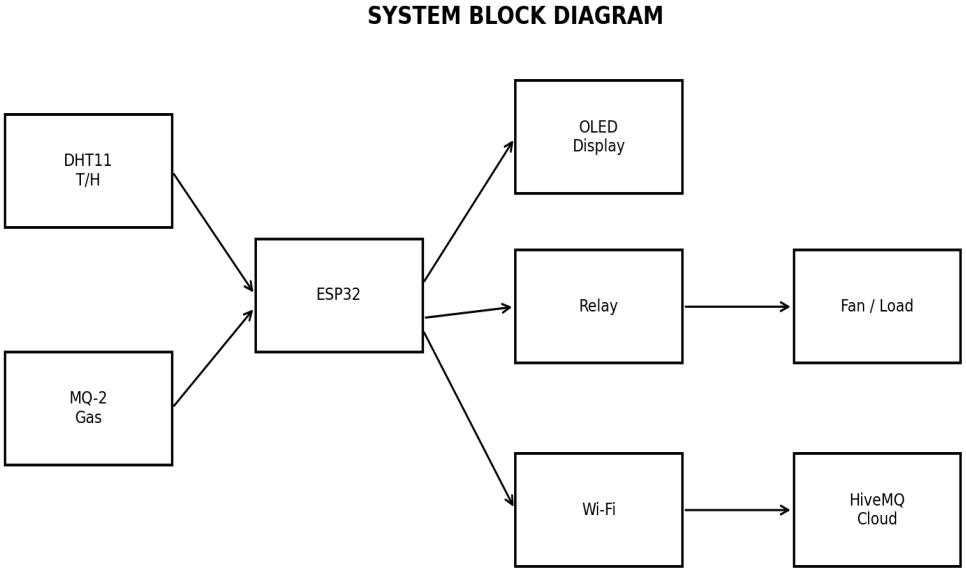

Figure 2.2 System block diagram

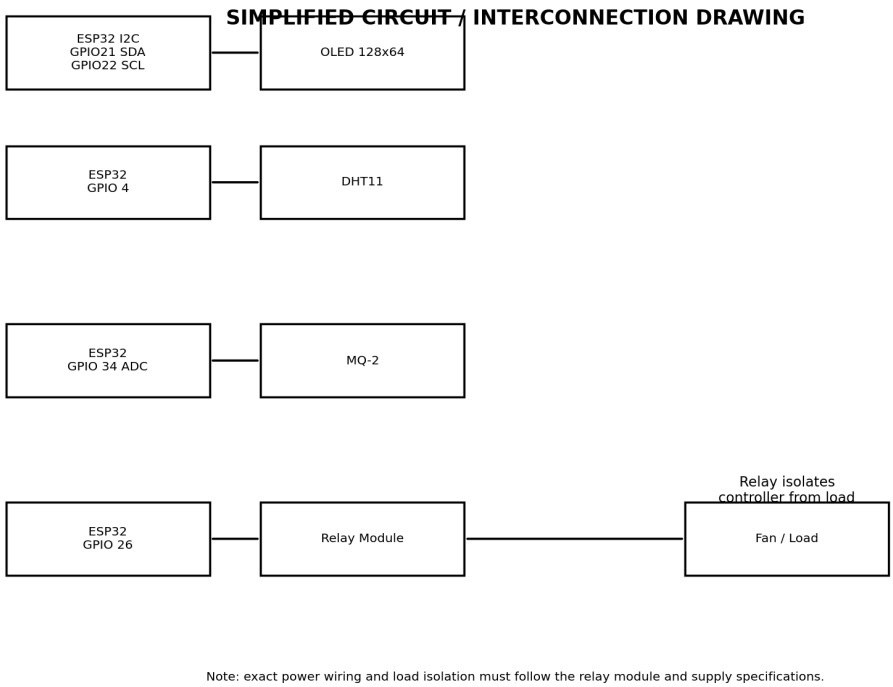

Figure 2.3 Simplified circuit/interconnection drawing

2.4 Documented ESP32 Pin Mapping

ESP32 pin	Device	Signal / function	Purpose
GPIO 4	DHT11 DATA	Digital input	Temperature / humidity acquisition
GPIO 34	MQ-2 AO	ADC input	Raw gas/smoke-related signal
GPIO 26	Relay IN	Digital output	Fan control
GPIO 21	OLED SDA	I2C data	Display communication
GPIO 22	OLED SCL	I2C clock	Display communication

2.5 MQTT Interface

Topic	Direction	Purpose	Example / command
industrial/site1/telemetry	ESP32 → broker	Sensor/state telemetry	{"temperature":29.5,"humidity":60.0,"gas":500,"fan":"OFF","mode":"AUTO"}
industrial/site1/command	Client → ESP32	Remote control	AUTO / ON / OFF
industrial/site1/status	ESP32 → broker	Availability	ESP32 ONLINE

3. HARDWARE & SOFTWARE REQUIREMENTS

Hardware	Role
ESP32 development board	Processing, Wi-Fi, GPIO and ADC
DHT11	Temperature and humidity sensing
MQ-2	Raw gas/smoke-related analog sensing
0.96-inch SSD1306 OLED	Local operating-state display
Relay module + demonstration fan/load	Actuation interface
Breadboard + jumper wires	Prototype interconnection

Software / library	Purpose
Arduino IDE	Compilation and firmware upload
ESP32 Arduino Core	ESP32 platform support
PubSubClient	MQTT client
WiFiClientSecure	TLS-capable network client
DHT library	DHT11 sensor interface
Adafruit GFX	Graphics support for OLED
Adafruit SSD1306	SSD1306 OLED interface
HiveMQ Cloud	Managed MQTT broker
Serial Monitor	Debugging and runtime observation

3.1 Configuration values that must be supplied by the evaluator/team

Parameter	Value in submission	Action
Wi-Fi SSID	YOUR_WIFI_NAME	Replace with the demonstration network name
Wi-Fi password	YOUR_WIFI_PASSWORD	Enter locally; do not commit to repository
HiveMQ host	Documented project broker host	Retain the project broker endpoint if still active
HiveMQ username	industrial_esp32	Use the valid project account if still active
HiveMQ password	YOUR_HIVEMQ_PASSWORD	Enter locally; do not publish
MQTT port	8883	TLS transport path used by the implementation
Serial baud	115200	Use in Arduino IDE Serial Monitor
OLED address	0x3C	Used by the firmware

Security note: The current implementation calls **setInsecure()**, so server certificate verification is not enforced. This is explicitly identified as a security limitation in the project record and should be corrected before real deployment.

4. EXPERIMENTAL SETUP

The documented experimental arrangement is a breadboard-based laboratory prototype containing the ESP32, DHT11, MQ-2, relay module, OLED, wiring and demonstration fan/load. The project record identifies the setup as a laboratory demonstrator rather than a certified industrial installation.

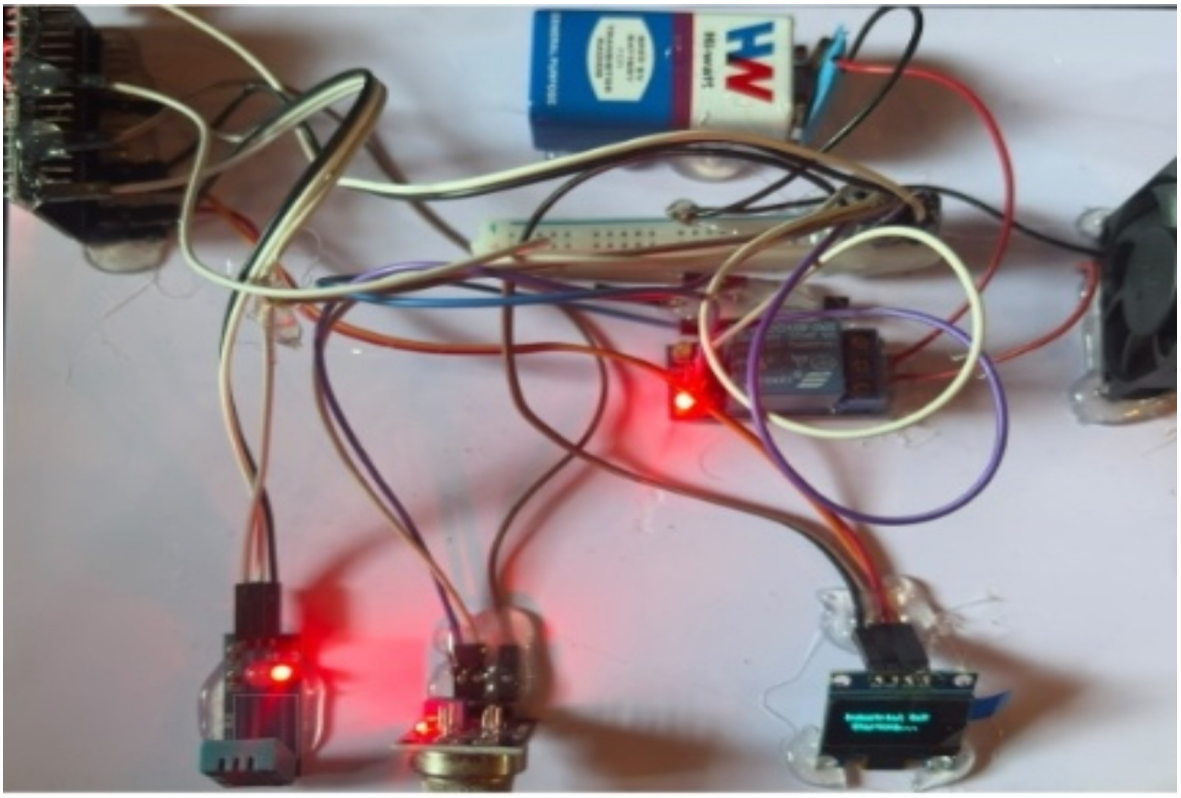

Figure 4.1 Actual implementation prototype supplied in the project record.

4.1 Hardware arrangement

- ESP32 acts as the central processing and communication node.
- DHT11 provides temperature and humidity input on GPIO 4.
- MQ-2 analog output is read on GPIO 34.
- Relay input is controlled from GPIO 26 and interfaces the controller to the demonstration fan/load.
- SSD1306 OLED uses I2C on GPIO 21/22 for local state display.
- Wi-Fi and HiveMQ Cloud provide the remote MQTT communication path.

4.2 Measurement and test conditions

- Environment: controlled laboratory demonstration.
- Inputs: ambient temperature/humidity and MQ-2 raw analog signal; MQTT commands for remote operation.
- Output observation: OLED display, Serial Monitor, relay/fan state and MQTT messages.
- Nominal sensor schedule: 3 seconds.
- Nominal MQTT retry schedule: approximately 5 seconds when disconnected.
- The supplied record contains one traceable OLED operating snapshot and functional test observations, but not a repeated numerical dataset.

4.3 Power supply documentation status

The supplied project evidence does not record the exact power-supply ratings, relay contact rating, load current or supply margin. These must be confirmed against the actual prototype before final submission. Do not invent or copy generic ratings into the final evidence package.

5. INSTRUCTIONS TO RUN THE PROJECT

These instructions are written so an evaluator familiar with ESP32 development can reproduce the demonstration. The team must replace the credential placeholders locally and must not publish passwords or private credentials.

Step 1 – Hardware Preparation

- Collect the ESP32 development board, DHT11, MQ-2, SSD1306 OLED, relay module, demonstration fan/load, breadboard and jumper wires.
- Wire the DHT11 DATA line to GPIO 4.
- Wire the MQ-2 analog output (AO) to GPIO 34.
- Wire the relay input to GPIO 26.
- Wire OLED SDA to GPIO 21 and OLED SCL to GPIO 22.
- Connect the prototype using the actual regulated supply arrangement used during the demonstration. Verify supply ratings and relay/load isolation before energizing the system.
- Connect the ESP32 to the development computer through its programming/debugging USB connection.

Step 2 – Software Installation

- Install Arduino IDE.
- Install/configure ESP32 board support through the ESP32 Arduino Core.
- Install the DHT library, PubSubClient, Adafruit GFX and Adafruit SSD1306 libraries.
- Use the source code included in Section 10 of this document.
- Open the Arduino project/source file and select the correct ESP32 board and COM port.

Step 3 – Configure the Project

- Replace YOUR_WIFI_NAME with the demonstration Wi-Fi SSID.
- Replace YOUR_WIFI_PASSWORD with the Wi-Fi password locally.
- Use the valid HiveMQ username/password locally.
- Do not place credentials in GitHub, screenshots, reports or shared folders.
- Keep the documented GPIO assignments and thresholds unless the actual prototype has been modified; if modified, update the final schematic and instructions.

Step 4 – Build / Upload / Execute

- Compile/verify the sketch in Arduino IDE.
- Upload the firmware to the ESP32.
- Open Serial Monitor at 115200 baud.
- Power the prototype and observe initialization.
- Confirm Wi-Fi connection and the printed IP/RSSI information.
- Confirm the ESP32 connects to HiveMQ Cloud and publishes ESP32 ONLINE.
- Observe the OLED for temperature, humidity, gas, fan state and mode.

Step 5 – Operate the System

- **AUTO:** publish AUTO to industrial/site1/command. The firmware evaluates temperature > 30 °C OR gas > 700 and controls the fan locally.
- **ON:** publish ON to the command topic. The firmware switches to manual mode and turns the fan ON.
- **OFF:** publish OFF to the command topic. The firmware switches to manual mode and turns the fan OFF.
- Observe the OLED, Serial Monitor, relay/fan and telemetry topic after each action.

Step 6 – Verify the Output

Use the following verification chain: **Input → Sensor/MQTT command → ESP32 processing → control decision → relay/fan or display → MQTT telemetry/status**. A successful demonstration should show consistent state information across the local OLED/Serial output and the MQTT observations.

6. EXPERIMENTAL DATA

The technical submission instructions require actual measured values rather than generic statements. The available project evidence contains one directly documented OLED snapshot. It is therefore presented as a single raw observation, not as a statistical performance dataset.

Trial	Temperature (°C)	Humidity (%)	MQ-2 ADC	Fan	Mode	Evidence
1	27.2	45.0	105	OFF	AUTO	OLED photograph

6.1 Processed data

Calculation	Value	Interpretation
Temperature threshold margin	30.0 – 27.2 = 2.8 °C	Observed temperature is 2.8 °C below the AUTO trigger
Gas threshold margin	700 – 105 = 595 ADC	Observed gas reading is 595 ADC below the AUTO trigger
Automatic decision	27.2 ≤ 30 and 105 ≤ 700	Both trigger conditions are false
Expected fan state	OFF	Matches the documented OLED state
Nominal sensing frequency	1 / 3 s = 0.333 Hz	Firmware scheduling frequency, not measured sensor response time

6.2 Data limitations

- No repeated dataset is available for mean, standard deviation, accuracy, percentage error or reliability rate.
- No repeated timestamps are available for a statistically meaningful MQTT latency or packet-loss calculation.
- The MQ-2 reading is raw ADC data and must not be converted to gas concentration without calibration.
- Exact electrical power consumption was not recorded, so no power-efficiency claim is made.

7. TESTING & VALIDATION

Test	Req.	Test item	Acceptance criterion	Evidence	Status
TC01	FR1	Power-on/startup	Firmware starts; relay initialized OFF	Serial / observation	Pass
TC02	FR2	DHT11 acquisition	Valid values obtained	OLED / Serial	Pass
TC03	FR3	MQ-2 acquisition	ADC value obtained	OLED / Serial	Pass
TC04	FR4	OLED display	T/RH/gas/fan/mode visible	OLED photo	Pass
TC05	FR5	AUTO threshold	Fan follows OR logic	Observed function	Pass
TC06	FR6	Telemetry	Telemetry payload available	MQTT observation	Pass
TC07	FR7–FR8	Remote commands	Mode/fan changes appropriately	MQTT observation	Pass
TC08	FR9	MQTT recovery	Firmware attempts reconnection	Recovery observation	Implemented
TC09	FR10	Status	ESP32 ONLINE published	MQTT observation	Pass

7.1 Verification interpretation

The supplied project record supports functional success for startup, Wi-Fi connection, DHT11 acquisition, MQ-2 acquisition, OLED update, AUTO control, manual ON/OFF commands, telemetry publication and online status. MQTT recovery is implemented in firmware, but the available record does not contain a repeated recovery dataset. Therefore it is marked **Implemented** rather than falsely reported as quantitatively validated.

Objective	Target	Achievement	Evidence basis
O1	DHT11 T/RH acquisition	Fully achieved	Functional evidence
O2	MQ-2 ADC on GPIO 34	Fully achieved	Functional evidence
O3	OLED status display	Fully achieved	OLED photograph
O4	Automatic fan control	Fully achieved	Functional test record
O5	Wi-Fi + HiveMQ MQTT	Fully achieved	Functional test record
O6	Telemetry publication	Fully achieved	Functional test record
O7	AUTO/ON/OFF commands	Fully achieved	Functional test record
O8	MQTT reconnection	Partially achieved	Implemented; repeated recovery dataset absent
O9	Quantitative performance evaluation	Partially achieved	Design parameters available; repeated measurements absent

8. PROJECT EVIDENCE & RESULTS

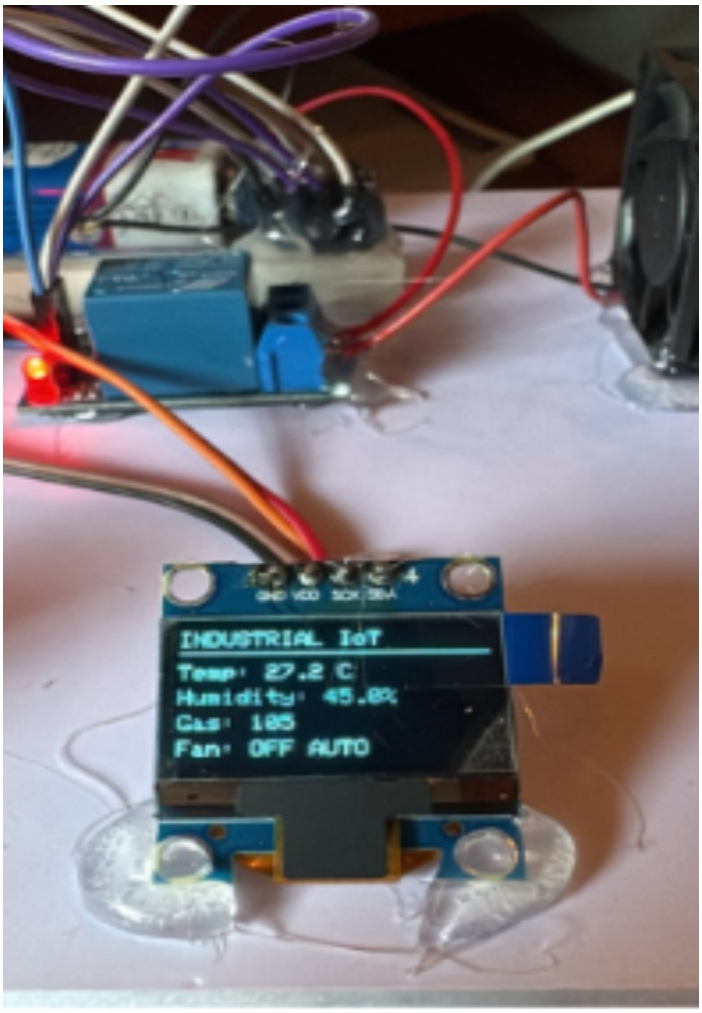

Figure 8.1 OLED operating state supplied in the project record.

The displayed state records **27.2 °C**, **45.0%** humidity, **MQ-2 = 105**, **Fan OFF** and **AUTO**. This is internally consistent with the control rule because neither the temperature nor gas threshold is exceeded.

8.1 Engineering significance

- The prototype demonstrates the complete embedded-to-cloud chain rather than only sensor logging.
- Local threshold control reduces dependence of basic actuation on cloud availability.
- MQTT provides separate telemetry, command and status paths.
- The OLED provides a local diagnostic interface useful when remote monitoring is unavailable.
- The current evidence supports functional demonstration but not industrial-grade performance claims.

8.2 Results that must NOT be claimed from the present evidence

- Sensor accuracy or calibrated gas concentration.
- Measured end-to-end MQTT latency distribution.
- Packet-loss percentage.
- Long-duration reliability or uptime.
- Certified industrial safety performance.
- Measured power efficiency.

9. SOFTWARE FLOW & OPERATING LOGIC

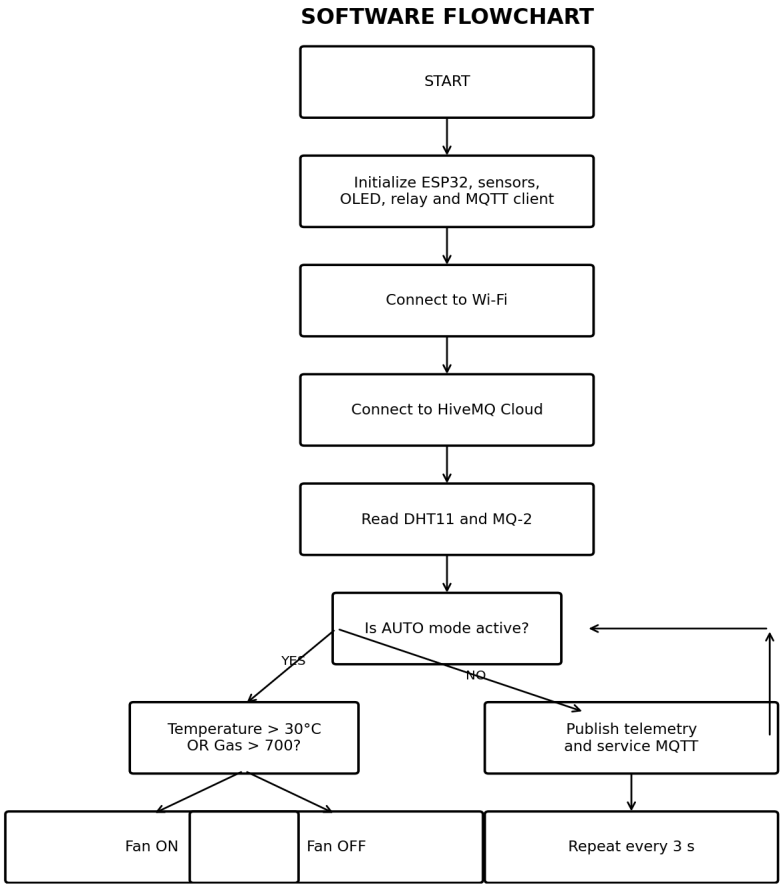

Figure 9.1 Software flowchart supplied in the project record.

The firmware initializes the ESP32 peripherals, connects to Wi-Fi and HiveMQ Cloud, reads the DHT11 and MQ-2 periodically, applies the AUTO threshold rule when enabled, updates the OLED/Serial Monitor, services MQTT traffic and publishes telemetry. MQTT commands can switch the controller between AUTO and manual ON/OFF modes.

Mode / condition	Firmware response
AUTO and T > 30 °C	Fan ON
AUTO and gas > 700 ADC	Fan ON
AUTO and neither threshold exceeded	Fan OFF
MQTT command ON	Manual mode; fan ON
MQTT command OFF	Manual mode; fan OFF
MQTT command AUTO	AUTO mode; thresholds immediately evaluated
MQTT disconnected	Reconnect attempt approximately every 5 s; local AUTO control remains available

10. GITHUB REPOSITORY / README CONTENT

The supplied submission instructions recommend a repository containing the final source, README, circuit/system diagrams, hardware/software material, test results, documentation and images. The following is the README content to place in the repository.

Project title: Industrial IoT Automation System Using ESP32 and HiveMQ Cloud

Project description: ESP32-based prototype for local environmental sensing, automatic fan control, OLED indication and MQTT remote monitoring/control through HiveMQ Cloud.

Problem statement: Provide low-cost remote visibility while retaining local automatic response during network/cloud interruption.

Objectives: Acquire DHT11 and MQ-2 signals; display status; implement AUTO threshold control; connect to Wi-Fi/HiveMQ; publish telemetry; receive AUTO/ON/OFF commands; handle MQTT reconnection; verify operation.

Hardware requirements: ESP32, DHT11, MQ-2, SSD1306 OLED, relay module, demonstration fan/load, breadboard and jumper wires.

Software requirements: Arduino IDE, ESP32 Arduino Core, DHT, PubSubClient, Adafruit GFX and Adafruit SSD1306 libraries.

Circuit/system diagram: Use the final diagram from Section 2 of this document.

Installation/setup: Install Arduino IDE and ESP32 support; install libraries; wire hardware according to GPIO mapping; enter credentials locally.

How to run: Select ESP32 board and COM port, compile, upload, open Serial Monitor at 115200 baud, power the prototype and verify Wi-Fi/MQTT connection.

How to operate: Send AUTO, ON or OFF to industrial/site1/command and observe OLED, relay/fan and telemetry.

Testing procedure: Execute TC01–TC09 from Section 7 and record actual observations.

Results: Functional tests pass as documented; quantitative performance remains under-validated because repeated datasets are absent.

Team members: Enter the final team member list approved by the team. Do not invent names or roles in this technical package.

References: Use the references already listed in the project report and the platform/library documentation used by the implementation.

Recommended repository structure

```
Industrial-IoT-Automation-System/
■■■ README.md
■■■ source_code/
■   ■■■ industrial_iot_clean.ino
■■■ circuit_diagram/
■   ■■■ system_architecture.png
■   ■■■ circuit_interconnection.png
■■■ hardware/
■■■ software/
■■■ dataset/
■■■ test_results/
■■■ documentation/
■■■ images/
```

11. COMPLETE SOURCE CODE

The source below is based on the project implementation. Credentials are placeholders. The code uses ESP32 Wi-Fi, WiFiClientSecure, PubSubClient, DHT, Wire, Adafruit GFX and Adafruit SSD1306. The implementation's setInsecure() call is retained as part of the documented prototype, but it must be replaced with proper certificate validation before real deployment.

```
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <PubSubClient.h>
#include <DHT.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

//
//
//
//
//
//
//
//
// ================================================
// WIFI
// ================================================

const char* WIFI_SSID = "YOUR_WIFI_NAME";
const char* WIFI_PASSWORD = "YOUR_WIFI_PASSWORD";

// ================================================
// HIVEMQ CLOUD
// ================================================

const char* MQTT_HOST =
    "fd439c9af8fd4b9586de13f67ead11e1.s1.eu.hivemq.cloud";

const int MQTT_PORT = 8883;

const char* MQTT_USERNAME = "industrial_esp32";
const char* MQTT_PASSWORD = "YOUR_HIVEMQ_PASSWORD";

// ================================================
// MQTT TOPICS
// ================================================

const char* TOPIC_TELEMETRY = "industrial/site1/telemetry";
const char* TOPIC_COMMAND = "industrial/site1/command";
const char* TOPIC_STATUS = "industrial/site1/status";

// ================================================
// PIN DEFINITIONS
// ================================================

#define DHT_PIN 4
#define DHT_TYPE DHT11

#define MQ2_PIN 34

#define RELAY_PIN 26

#define OLED_SDA 21
#define OLED_SCL 22

// ================================================
// OBJECTS
// ================================================

DHT dht(DHT_PIN, DHT_TYPE);

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

Adafruit_SSD1306 display(
    SCREEN_WIDTH,
    SCREEN_HEIGHT,
    &Wire,
    -1
);

WiFiClientSecure secureClient;
PubSubClient mqttClient(secureClient);
```

11. COMPLETE SOURCE CODE (continued)

```
// ====================================================
// VARIABLES
// ====================================================

float temperature = 0;
float humidity = 0;

int gasValue = 0;

bool fanState = false;

// AUTO mode initially
bool autoMode = true;

// ====================================================
// THRESHOLDS
// ====================================================

// Fan ON when temperature is ABOVE 30°C
const float FAN_ON_TEMP = 30.0;

// Fan ON when MQ-2 ADC reading is ABOVE 700
const int GAS_THRESHOLD = 700;

// ====================================================
// TIMING
// ====================================================

unsigned long lastSensorRead = 0;
unsigned long lastMQTTReconnect = 0;

const unsigned long SENSOR_INTERVAL = 3000;

// ====================================================
// RELAY CONTROL
// ====================================================

// YOUR RELAY IS REVERSED
//
// HIGH = FAN ON
// LOW  = FAN OFF

void fanON()
{
    digitalWrite(RELAY_PIN, HIGH);
    fanState = true;
}

void fanOFF()
{
    digitalWrite(RELAY_PIN, LOW);
    fanState = false;
}

// ====================================================
// OLED
// ====================================================

void updateOLED()
{
    display.clearDisplay();

    display.setTextSize(1);
    display.setTextColor(SSD1306_WHITE);

    display.setCursor(0, 0);
    display.println("INDUSTRIAL IoT");

    display.drawLine(
        0, 10,
        127, 10,
        SSD1306_WHITE
    );
```

11. COMPLETE SOURCE CODE (continued)

```
    display.setCursor(0, 16);
    display.print("Temp: ");
    display.print(temperature, 1);
    display.println(" C");

    display.setCursor(0, 28);
    display.print("Humidity: ");
    display.print(humidity, 1);
    display.println("%");

    display.setCursor(0, 40);
    display.print("Gas: ");
    display.println(gasValue);

    display.setCursor(0, 52);
    display.print("Fan: ");
    display.print(fanState ? "ON" : "OFF");

    display.print(" ");
    display.print(autoMode ? "AUTO" : "MAN");

    display.display();
}

// ==================================================
// READ SENSORS
// ==================================================

void readSensors()
{
    float newTemperature =
        dht.readTemperature();

    float newHumidity =
        dht.readHumidity();

    if (!isnan(newTemperature))
    {
        temperature = newTemperature;
    }

    if (!isnan(newHumidity))
    {
        humidity = newHumidity;
    }

    // Read MQ-2
    gasValue = analogRead(MQ2_PIN);

    // ==================================================
    // AUTOMATIC FAN CONTROL
    // ==================================================

    if (autoMode)
    {
        // FAN ON if EITHER condition is true
        //
        // Temperature > 30°C
        // OR
        // Gas > 700

        if (
            temperature > FAN_ON_TEMP ||
            gasValue > GAS_THRESHOLD
        )
        {
            fanON();
        }
        else
        {
            fanOFF();
        }
    }

    updateOLED();
```

11. COMPLETE SOURCE CODE (continued)

```
    Serial.println();
    Serial.println("==============================");

    Serial.print("Temperature: ");
    Serial.print(temperature);
    Serial.println(" C");

    Serial.print("Humidity: ");
    Serial.print(humidity);
    Serial.println(" %");

    Serial.print("MQ-2: ");
    Serial.println(gasValue);

    Serial.print("Fan: ");
    Serial.println(
        fanState ? "ON" : "OFF"
    );

    Serial.print("Mode: ");
    Serial.println(
        autoMode ? "AUTO" : "MANUAL"
    );

    Serial.println("==============================");
}

// ======================================================
// SEND MQTT TELEMETRY
// ======================================================

void publishTelemetry()
{
    char payload[300];

    snprintf(
        payload,
        sizeof(payload),
        "{\"temperature\":%.2f,"
        "\"humidity\":%.2f,"
        "\"gas\":%d,"
        "\"fan\":\"%s\","
        "\"mode\":\"%s\"}",
        temperature,
        humidity,
        gasValue,
        fanState ? "ON" : "OFF",
        autoMode ? "AUTO" : "MANUAL"
    );

    mqttClient.publish(
        TOPIC_TELEMETRY,
        payload
    );

    Serial.println("Telemetry sent:");
    Serial.println(payload);
}

// ======================================================
// MQTT COMMAND RECEIVER
// ======================================================

void mqttCallback(
    char* topic,
    byte* payload,
    unsigned int length
)
{
    String command = "";

    for (
        unsigned int i = 0;
        i < length;
```

11. COMPLETE SOURCE CODE (continued)

```
        i++
    )
    {
        command += (char)payload[i];
    }

    command.trim();
    command.toUpperCase();

    Serial.print("MQTT Command: ");
    Serial.println(command);

    // ================================================
    // AUTO
    // ================================================

    if (command == "AUTO")
    {
        autoMode = true;

        // Apply both automatic thresholds immediately

        if (
            temperature > FAN_ON_TEMP ||
            gasValue > GAS_THRESHOLD
        )
        {
            fanON();
        }
        else
        {
            fanOFF();
        }
    }

    // ================================================
    // MANUAL FAN ON
    // ================================================

    else if (command == "ON")
    {
        autoMode = false;

        fanON();
    }

    // ================================================
    // MANUAL FAN OFF
    // ================================================

    else if (command == "OFF")
    {
        autoMode = false;

        fanOFF();
    }

    updateOLED();

    // Publish immediate status
    publishTelemetry();

    Serial.print("Mode: ");
    Serial.println(
        autoMode ? "AUTO" : "MANUAL"
    );

    Serial.println("==============================");
}

// ======================================================
// CONNECT TO WIFI
// ======================================================

void connectWiFi()
```

11. COMPLETE SOURCE CODE (continued)

```
{
    Serial.println();
    Serial.print("Connecting to Wi-Fi");

    WiFi.begin(
        WIFI_SSID,
        WIFI_PASSWORD
    );

    while (
        WiFi.status() != WL_CONNECTED
    )
    {
        delay(500);
        Serial.print(".");
    }

    Serial.println();
    Serial.println("Wi-Fi Connected!");

    Serial.print("IP Address: ");
    Serial.println(WiFi.localIP());

    Serial.print("Signal Strength: ");
    Serial.print(WiFi.RSSI());
    Serial.println(" dBm");
}

// ======================================================
// CONNECT TO HIVEMQ
// ======================================================

void connectMQTT()
{
    if (mqttClient.connected())
    {
        return;
    }

    Serial.println();
    Serial.println("Connecting to HiveMQ...");

    String clientID =
        "ESP32_Industrial_"
        + String(
            (uint32_t)ESP.getEfuseMac(),
            HEX
        );

    if (
        mqttClient.connect(
            clientID.c_str(),
            MQTT_USERNAME,
            MQTT_PASSWORD
        )
    )
    {
        Serial.println(
            "HiveMQ Connected!"
        );

        // Subscribe to commands
        mqttClient.subscribe(
            TOPIC_COMMAND
        );

        mqttClient.publish(
            TOPIC_STATUS,
            "ESP32 ONLINE"
        );

        publishTelemetry();
    }
    else
    {
```

11. COMPLETE SOURCE CODE (continued)

```
        Serial.print(
            "MQTT connection failed. State: "
        );

        Serial.println(
            mqttClient.state()
        );
    }
}

// ====================================================
// SETUP
// ====================================================

void setup()
{
    Serial.begin(115200);

    delay(1000);

    Serial.println();

    Serial.println(
        "================================="
    );

    Serial.println(
        " INDUSTRIAL IoT AUTOMATION"
    );

    Serial.println(
        " ESP32 + HiveMQ Cloud"
    );

    Serial.println(
        "================================="
    );

    // ================================================
    // RELAY
    // ================================================

    pinMode(
        RELAY_PIN,
        OUTPUT
    );

    // Start with fan OFF
    fanOFF();

    // ================================================
    // MQ2
    // ================================================

    pinMode(
        MQ2_PIN,
        INPUT
    );

    // ================================================
    // DHT
    // ================================================

    dht.begin();

    // ================================================
    // OLED
    // ================================================

    Wire.begin(
        OLED_SDA,
        OLED_SCL
    );

    if (
```

11. COMPLETE SOURCE CODE (continued)

```
        !display.begin(
            SSD1306_SWITCHCAPVCC,
            0x3C
        )
    )
    {
        Serial.println(
            "OLED not found!"
        );
    }
    else
    {
        display.clearDisplay();

        display.setTextSize(1);

        display.setTextColor(
            SSD1306_WHITE
        );

        display.setCursor(10, 20);

        display.println(
            "Industrial IoT"
        );

        display.setCursor(20, 35);

        display.println(
            "Starting..."
        );

        display.display();

        delay(2000);
    }

    // ================================================
    // Wi-Fi
    // ================================================

    connectWiFi();

    // ================================================
    // TLS
    // ================================================

    secureClient.setInsecure();

    // ================================================
    // MQTT
    // ================================================

    mqttClient.setServer(
        MQTT_HOST,
        MQTT_PORT
    );

    mqttClient.setCallback(
        mqttCallback
    );

    mqttClient.setBufferSize(
        512
    );

    // ================================================
    // CONNECT MQTT
    // ================================================

    connectMQTT();
}

// ================================================
// LOOP
```

11. COMPLETE SOURCE CODE (continued)

```
// ====================================================

void loop()
{
    // ================================================
    // KEEP MQTT CONNECTION ALIVE
    // ================================================

    if (!mqttClient.connected())
    {
        unsigned long now =
            millis();

        if (
            now - lastMQTTReconnect > 5000
        )
        {
            lastMQTTReconnect = now;

            connectMQTT();
        }
    }
    else
    {
        mqttClient.loop();
    }

    // ================================================
    // READ SENSORS PERIODICALLY
    // ================================================

    unsigned long now =
        millis();

    if (
        now - lastSensorRead >=
        SENSOR_INTERVAL
    )
    {
        lastSensorRead = now;

        readSensors();

        if (mqttClient.connected())
        {
            publishTelemetry();
        }
    }
}
```

12. FINAL SUBMISSION LINKS & CHECKLIST

Submission item	Final value
GitHub Repository	TO BE ADDED – paste the accessible final repository link
Project Demonstration Video	TO BE ADDED – paste the accessible Google Drive video link

The supplied instructions require both links to be tested from a different browser/account or an incognito window before submission.

Status	Checklist item
✓	Final circuit diagram matches the actual prototype
✓	Complete source code included
■	GitHub repository accessible and final version uploaded
✓	README/setup content prepared
✓	Hardware/software requirements documented
✓	Experimental setup photograph included
✓	Step-by-step execution instructions included
✓	Requirement-linked test plan included
✓	Available raw experimental data included
✓	Processed calculations included
✓	Results and validation tables included
—	Graph omitted because the available dataset contains only one traceable operating snapshot
✓	Verification/validation status included
✓	Prototype/OLED evidence included
■	Video demonstration link added and checked

FINAL WARNING BEFORE SUBMISSION

Do not submit the document with invented power ratings, fabricated repeated measurements, fake GitHub/video links, or unsupported sensor-accuracy claims. The project record itself identifies these as evidence gaps. Fill only the fields that can be verified from the actual prototype and team records.

APPENDIX A – QUICK REPRODUCTION CARD

Item	Value
Controller	ESP32
DHT11 DATA	GPIO 4
MQ-2 AO	GPIO 34
Relay IN	GPIO 26
OLED SDA/SCL	GPIO 21 / GPIO 22
OLED address	0x3C
MQTT port	8883
Sensor interval	3000 ms
MQTT retry	5000 ms nominal
AUTO temperature trigger	> 30 °C
AUTO gas trigger	> 700 ADC
Commands	AUTO / ON / OFF
Telemetry topic	industrial/site1/telemetry
Command topic	industrial/site1/command
Status topic	industrial/site1/status
Serial Monitor	115200 baud

Source basis: the submitted individual capstone report, its embedded prototype diagrams/photos and source-code appendix, plus the supplied Capstone Project – Technical Submission Instructions. The technical package intentionally distinguishes documented implementation from missing experimental evidence.